

Resume Materi Perkuliahan
Perancangan dan Analisis Algoritma (A)
Pertemuan ke-7

Disusun Oleh:
Gilbran Mahdavia Raja
NRP: 5025241134

Institut Teknologi Sepuluh Nopember
Surabaya
2026

Daftar Isi

1	Pendahuluan	2
1.1	Latar Belakang	2
1.2	Perbedaan Paradigma Kedua Masalah	2
1.3	Tujuan Laporan	2
2	Landasan Teori	3
2.1	Rekurensi dan Pemrograman Dinamis	3
2.2	Solusi <i>Closed-Form</i>	3
2.3	Partisi Bilangan Bulat	3
2.4	Graf Keadaan dan Jalur Terpendek	3
3	Studi Kasus 1: SPOJ EIDSALAMI – Eid Salami	4
3.1	Deskripsi Masalah	4
3.2	Definisi Formal	4
3.3	Derivasi Rekurensi	4
3.4	Derivasi Matematis: Dari Rekurensi ke <i>Closed-Form</i>	5
3.4.1	Kasus $k = 1$ dan $k = 2$	5
3.4.2	Kasus $k = 3$: Kemunculan Fungsi Kosinus	5
3.4.3	Kasus $k = 4$: Kompleksitas yang Bertambah	5
3.5	Dua Pendekatan Solusi	5
3.5.1	Pendekatan A: <i>Closed-Form</i> (Floating Point)	5
3.5.2	Pendekatan B: Dynamic Programming (Integer Eksak) — Kode yang Disubmit	6
3.6	Analisis Kompleksitas Komparatif	7
4	Studi Kasus 2: SPOJ MST1 – Minimum Step To One	7
4.1	Deskripsi Masalah	7
4.2	Definisi Formal	8
4.3	Mengapa <i>Greedy</i> Gagal?	8
4.4	Pemodelan DP	8
4.5	Pendekatan <i>Hybrid: Bottom-Up + Top-Down</i>	9
4.5.1	Motivasi Pendekatan <i>Hybrid</i>	9
4.5.2	Keamanan Rekursi untuk $n > 50.000$	9
4.5.3	Implementasi C++ yang Disubmit	9
4.6	Analisis Kompleksitas	10
5	Perbandingan dan Analisis Dua Studi Kasus	10
5.1	Tabel Komparatif	10
5.2	Narasi Perbandingan	10
6	Kesimpulan	11
6.1	Pola Berpikir Sistematis dalam Memecahkan Masalah DP	11
6.2	Ringkasan Konsep Kunci	12
7	Referensi	13
8	Bukti Submission	14

1 Pendahuluan

1.1 Latar Belakang

Pemrograman kompetitif merupakan salah satu wahana paling efektif untuk menginternalisasi konsep-konsep fundamental algoritma. Setiap soal memaksa pemecah masalah untuk tidak sekadar memahami definisi, tetapi juga menemukan pola, membangun abstraksi, dan mengimplementasikan solusi secara efisien dalam batasan sumber daya yang ketat.

Laporan ini menganalisis dua soal dari platform SPOJ (*Sphere Online Judge*) yang dipilih secara sengaja karena mewakili dua kutub paradigma yang berbeda dalam kompetisi pemrograman:

1. **SPOJ EIDSALAMI – Eid Salami:** Soal ini berakar pada kombinatorika matematika, khususnya teori *integer partition*. Solusinya melibatkan derivasi *closed-form* dari rekurensi, dengan fenomena menarik bahwa ekspresi bilangan bulat pada akhirnya mengandung fungsi trigonometri periodik — suatu hasil yang kontra-intuitif namun dapat dijelaskan secara *rigorous* melalui teori fungsi pembangkit.
2. **SPOJ MST1 – Minimum Step To One:** Soal ini merupakan masalah optimasi klasik berbasis *dynamic programming* (DP). Meski tampak sederhana, batasan nilai n yang besar menuntut pendekatan *hybrid* — memadukan *bottom-up* DP untuk nilai kecil dengan rekursi *top-down* berkedalaman $O(\log n)$ untuk nilai besar.

1.2 Perbedaan Paradigma Kedua Masalah

Perbedaan mendasar antara EIDSALAMI dan MST1 terletak pada sifat matematika yang mendasarinya:

- **EIDSALAMI** bersifat matematis-kombinatorial. Solusi optimal tidak dihasilkan dengan menelusuri ruang keadaan, melainkan dengan menemukan pola aljabar dari barisan rekursif dan mengekspresikannya dalam bentuk tertutup. Pendekatan ini mereduksi kompleksitas komputasi dari $O(n \cdot k)$ menjadi $O(1)$ per kueri.
- **MST1** bersifat optimasi berbasis keadaan. Setiap nilai n adalah sebuah *state*, dan transisi antar *state* mengikuti tiga operasi yang terdefinisi secara eksplisit. Masalah ini dapat dimodelkan sebagai pencarian jalur terpendek pada graf berarah berbobot seragam.

1.3 Tujuan Laporan

Laporan ini tidak sekadar menyajikan solusi akhir, melainkan bertujuan untuk:

1. Merekonstruksi alur penalaran di balik setiap langkah solusi, mulai dari observasi awal, pola yang ditemukan, hingga generalisasi formal.
2. Menunjukkan bagaimana abstraksi matematis dapat menyederhanakan masalah yang tampak kompleks.
3. Membandingkan secara kritis dua paradigma penyelesaian masalah dalam konteks pemrograman kompetitif beserta *trade-off* masing-masing.

2 Landasan Teori

2.1 Rekurensi dan Pemrograman Dinamis

Definisi 2.1 (Relasi Rekurensi). Suatu relasi rekurensi adalah persamaan yang mendefinisikan suku ke- n dari suatu barisan berdasarkan satu atau lebih suku sebelumnya. Secara formal, $a_n = f(a_{n-1}, a_{n-2}, \dots, a_{n-k})$ untuk suatu fungsi f dan bilangan bulat positif k .

Definisi 2.2 (Pemrograman Dinamis). Pemrograman dinamis (DP) adalah paradigma algoritma yang memecah masalah menjadi submasalah yang saling tumpang-tindih (*overlapping subproblems*) dan menyimpan hasilnya (*memoization* atau *tabulation*) untuk menghindari komputasi berulang.

Prinsip Optimalitas Bellman

Suatu solusi optimal dari sebuah masalah selalu mengandung solusi optimal dari submasalah-submasalahnya. Prinsip inilah yang membuat DP valid sebagai strategi pemecahan masalah optimasi.

2.2 Solusi *Closed-Form*

Definisi 2.3 (Solusi *Closed-Form*). Sebuah ekspresi dikatakan *closed-form* apabila nilainya dapat dihitung dalam sejumlah langkah operasi matematika elementer yang terbatas (*penjumlahan, perkalian, pangkat, fungsi trigonometri, dan lain-lain*) tanpa melibatkan batas jumlah tak hingga atau iterasi eksplisit.

Solusi *closed-form* memiliki keunggulan waktu komputasi $O(1)$ per kueri, namun berpotensi mengalami *floating-point error* apabila melibatkan bilangan irasional seperti $\sqrt{3}$ atau fungsi trigonometri. Oleh karena itu, validasi hasil melalui tabulasi manual tetap diperlukan sebelum implementasi.

2.3 Partisi Bilangan Bulat

Definisi 2.4 (Fungsi Partisi). Sebuah partisi dari bilangan bulat positif n adalah cara menuliskan n sebagai jumlah bilangan bulat positif tanpa memperhatikan urutan. Fungsi $p(n)$ menyatakan jumlah partisi dari n , sedangkan $P(n, k)$ menyatakan jumlah partisi dari n menjadi tepat k bagian.

Sebagai contoh, $P(5, 2) = 2$ karena $5 = 4 + 1 = 3 + 2$ (dua cara yang mungkin).

2.4 Graf Keadaan dan Jalur Terpendek

Banyak masalah DP dapat dimodelkan sebagai pencarian jalur terpendek pada *directed acyclic graph* (DAG) atau graf berbobot. Dalam konteks MST1, setiap *state* (nilai n) adalah simpul, dan setiap operasi yang valid adalah busur berbobot 1. Pemodelan ini membuka kemungkinan penggunaan algoritma BFS, Dijkstra, maupun DP untuk mencari solusi.

3 Studi Kasus 1: SPOJ EIDSALAMI – Eid Salami

3.1 Deskripsi Masalah

Dalam perayaan Idul Fitri, seorang paman ingin membagikan salam tempel kepada sejumlah keponakan dengan aturan bahwa keponakan yang lebih tua harus menerima jumlah yang lebih besar dari yang lebih muda (secara ketat). Masalah ini meminta perhitungan banyaknya cara membagikan M unit uang kepada k orang dengan distribusi bertingkat.

Secara matematis, ini ekuivalen dengan: *berapa banyak cara menyatakan bilangan bulat M sebagai jumlah dari tepat k bilangan bulat positif yang berbeda-beda?*

Dengan transformasi variabel $n = M - \binom{k}{2}$ — yang berfungsi mengeliminasi syarat “berbeda” — masalah ini tereduksi menjadi menghitung $P(n, k)$, yaitu jumlah cara mempartisi n menjadi tepat k bagian.

3.2 Definisi Formal

Definisi 3.1 (Fungsi Partisi $P(n, k)$). $P(n, k)$ didefinisikan sebagai jumlah cara menyatakan n sebagai

$$n = x_1 + x_2 + \cdots + x_k,$$

dengan syarat $x_i \geq 1$ untuk semua i dan $x_1 \leq x_2 \leq \cdots \leq x_k$ (urutan tidak diperhatikan). Kondisi batas: $P(n, 1) = 1 \ \forall n \geq 1$, dan $P(n, k) = 0$ jika $k > n$ atau $k \leq 0$.

3.3 Derivasi Rekurensi

Setiap partisi dari n menjadi k bagian dapat diklasifikasikan berdasarkan nilai bagian terkecilnya.

- **Kasus 1 — Bagian terkecil bernilai 1:** Hapus bagian tersebut, sehingga tersisa partisi dari $n - 1$ menjadi $k - 1$ bagian. Kontribusi: $P(n - 1, k - 1)$.
- **Kasus 2 — Semua bagian bernilai ≥ 2 :** Kurangi setiap bagian dengan 1, sehingga diperoleh partisi dari $n - k$ menjadi k bagian. Kontribusi: $P(n - k, k)$.

Rekurensi Partisi

$$P(n, k) = P(n - 1, k - 1) + P(n - k, k)$$

dengan kondisi batas $P(n, n) = P(n, 1) = 1$ dan $P(n, k) = 0$ untuk $k > n$ atau $n \leq 0$.

Rekurensi ini langsung mengarah pada solusi DP dua dimensi dengan spesifikasi:

- **State:** $PF[i][j] = P(i, j)$
- **Transisi:** $PF[i][j] = PF[i - 1][j - 1] + PF[i - j][j]$
- **Basis:** $PF[i][i] = 1$; nilai nol untuk $j > i$ atau $i = 0$
- **Ukuran tabel:** $N \times K$ dengan $N = 10^6$, $K = 4$

3.4 Derivasi Matematis: Dari Rekurensi ke *Closed-Form*

3.4.1 Kasus $k = 1$ dan $k = 2$

Untuk $k = 1$, secara langsung $P(n, 1) = 1$ untuk semua $n \geq 1$.

Untuk $k = 2$, kita menghitung cara menyatakan $n = a + b$ dengan $1 \leq a \leq b$:

$$P(n, 2) = \left\lfloor \frac{n}{2} \right\rfloor = \frac{1}{4} [2n - 1 + (-1)^n].$$

Suku $(-1)^n$ muncul untuk menyatukan kasus genap dan ganjil dalam satu ekspresi.

3.4.2 Kasus $k = 3$: Kemunculan Fungsi Kosinus

$P(n, 3)$ — *Closed-Form*

$$P(n, 3) = \frac{1}{72} \left[6n^2 - 7 - 9(-1)^n + 16 \cos\left(\frac{2\pi n}{3}\right) \right]$$

Fungsi $\cos\left(\frac{2\pi n}{3}\right)$ muncul dari *partial fraction decomposition* fungsi pembangkit:

$$\sum_{n=0}^{\infty} P(n, 3) x^n = \frac{x^3}{(1-x)(1-x^2)(1-x^3)}.$$

Penyebut $(1-x^3)$ memiliki akar-akar $x = 1, e^{2\pi i/3}, e^{-2\pi i/3}$. Kontribusi dari pasangan akar kompleks $e^{\pm 2\pi i/3}$ menghasilkan koefisien berbentuk:

$$A \cdot (e^{2\pi i n/3} + e^{-2\pi i n/3}) = 2A \cos\left(\frac{2\pi n}{3}\right).$$

Catatan. Meski formula $P(n, 3)$ mengandung $\cos\left(\frac{2\pi n}{3}\right)$, hasilnya selalu berupa bilangan bulat untuk n bulat karena $\cos\left(\frac{2\pi n}{3}\right) \in \{1, -\frac{1}{2}\}$ dan konstanta $\frac{16}{72}$ dirancang sedemikian sehingga hasil akhir selalu bulat. Secara praktis, nilai $16 \cos\left(\frac{2\pi n}{3}\right)$ bergantung pada $n \bmod 3$: bernilai $+16$ jika $n \equiv 0$, dan -8 jika $n \equiv 1$ atau $n \equiv 2$.

3.4.3 Kasus $k = 4$: Kompleksitas yang Bertambah

$$P(n, 4) = \frac{1}{864} \left\{ n^3(n+1)2n(n+2) - 13 + 9(-1)^n - 96 \cos\left(\frac{2\pi n}{3}\right) + 32\sqrt{3} \sin\left(\frac{2\pi n}{3}\right) \right\}.$$

Kemunculan $\sin\left(\frac{2\pi n}{3}\right)$ dan $\sqrt{3}$ pada $k = 4$ berasal dari kontribusi akar kompleks primitif ketiga $e^{\pm 2\pi i/3}$ dalam *partial fraction decomposition* fungsi pembangkit $\frac{x^{10}}{(1-x)(1-x^2)(1-x^3)(1-x^4)}$.

3.5 Dua Pendekatan Solusi

3.5.1 Pendekatan A: *Closed-Form* (Floating Point)

Menggunakan formula $P(n, 3)$ secara langsung:

```
1 #include <cmath>
2 #include <cstdio>
3 using namespace std;
4 #define PI acos(-1.0)
```

```

5
6 int main() {
7     int cases, cid = 1;
8     double M;
9     scanf("%d", &cases);
10    while (cases--) {
11        scanf("%lf", &M);
12        // n = M - 3 (transformasi offset soal)
13        printf("Case %d: %.01f\n", cid++,
14              floor(((6.0*pow(M-3,2.0)) - 7.0
15                    - (9.0*pow(-1.0, M-3))
16                    + (16.0*cos(2.0/3.0*PI*(M-3)))) / 72.0));
17    }
18    return 0;
19 }

```

Listing 1: Solusi EIDSALAMI — *Closed-Form*

Kelemahan pendekatan ini adalah potensi *floating-point error* untuk n besar, karena fungsi `cos` dan `pow` menggunakan presisi ganda 64-bit. Meskipun `floor()` membantu, pendekatan ini tidak menjamin kebenaran pada semua kasus ketika hasil berada sangat dekat dengan bilangan bulat.

3.5.2 Pendekatan B: Dynamic Programming (Integer Eksak) — Kode yang Disubmit

```

1 #include <cstdio>
2 #include <cstring>
3 #include <algorithm>
4 using namespace std;
5 #define MAXN 1000001
6 #define MAXK 4
7
8 unsigned long long PF[MAXN][MAXK];
9
10 void pre_calc() {
11     int i, j;
12     memset(PF, 0, sizeof(PF));
13     for (i = 0; i < MAXN; i++) {
14         for (j = 1; j < MAXK; j++) {
15             if (j == i)
16                 PF[i][j] = 1;
17             else if (j < i)
18                 PF[i][j] = PF[i-1][j-1] + PF[i-j][j];
19         }
20     }
21 }
22
23 int main() {
24     int cases, cid = 1, M;
25     pre_calc();
26     if (scanf("%d", &cases) != EOF) {

```

```

27     while (cases--) {
28         if (scanf("%d", &M) != EOF) {
29             // Indeks M-3: offset sesuai transformasi soal
30             printf("Case %d: %llu\n", cid++, PF[M-3][3]);
31         }
32     }
33 }
34 return 0;
35 }

```

Listing 2: Solusi EIDSALAMI — DP *Precompute* (kode submit)

3.6 Analisis Kompleksitas Komparatif

Tabel 1: Perbandingan Kompleksitas Dua Pendekatan EIDSALAMI

Aspek	<i>Closed-Form</i>	DP <i>Precompute</i>
Waktu setup	$O(1)$	$O(N \cdot K)$
Waktu per kueri	$O(1)$	$O(1)$
Memori	$O(1)$	$O(N \cdot K)$
Akurasi	Floating (rentan galat)	Bilangan bulat eksak
Skalabilitas	Terbatas ($k \leq 4$)	Fleksibel

Korelasi Rekurensi dan *Closed-Form*

Rekurensi $P(n, k) = P(n-1, k-1) + P(n-k, k)$ dan *closed-form* $P(n, 3)$ bukanlah dua objek yang berbeda — keduanya adalah representasi yang berbeda dari objek matematika yang sama. *Closed-form* dapat diturunkan dari rekurensi melalui tiga tahap: (1) membangun *generating function*, (2) *partial fraction decomposition*, dan (3) mengekstrak koefisien. DP dan *closed-form* bukan pendekatan yang berlawanan, melainkan dua solusi yang sah dengan *trade-off* yang berbeda.

4 Studi Kasus 2: SPOJ MST1 – Minimum Step To One

4.1 Deskripsi Masalah

Diberikan bilangan bulat positif n . Pada setiap langkah, kita dapat melakukan salah satu dari tiga operasi berikut:

- (a) Pengurangan: $n \leftarrow n - 1$
- (b) Pembagian dua (jika n habis dibagi 2): $n \leftarrow n/2$
- (c) Pembagian tiga (jika n habis dibagi 3): $n \leftarrow n/3$

Pertanyaan: berapa jumlah langkah minimum untuk mengubah n menjadi 1?

Contoh ilustratif:

- $n = 4$: $4 \rightarrow 2 \rightarrow 1$ (2 langkah)

- $n = 7$: $7 \rightarrow 6 \rightarrow 3 \rightarrow 1$ (3 langkah)
- $n = 10$: Pendekatan *greedy* memberikan $10 \rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 1$ (4 langkah), sedangkan jalur optimal adalah $10 \rightarrow 9 \rightarrow 3 \rightarrow 1$ (3 langkah).

Contoh terakhir menunjukkan bahwa pemilihan operasi yang tampak paling menguntungkan secara lokal belum tentu menghasilkan solusi global yang optimal.

4.2 Definisi Formal

Fungsi Langkah Minimum $f(n)$

$$f(n) = \min\{1 + f(n-1), [2 \mid n] \cdot (1 + f(n/2)), [3 \mid n] \cdot (1 + f(n/3))\}$$

dengan kondisi batas $f(1) = 0$. Notasi $[P]$ bernilai 1 jika pernyataan P benar, dan operasi yang tidak memenuhi syarat habis bagi diabaikan dari komputasi minimum.

4.3 Mengapa *Greedy* Gagal?

Kegagalan *Greedy* pada MST1

Pendekatan *greedy* tidak menghasilkan solusi optimal untuk MST1. Kegagalan ini berakar pada ketiadaan *greedy choice property*: pilihan lokal terbaik (operasi dengan reduksi nilai terbesar) tidak selalu mengarah pada solusi global terbaik. DP menjamin optimalitas karena mempertimbangkan seluruh pilihan valid dan memanfaatkan *optimal substructure*.

Masalah MST1 dapat dimodelkan sebagai graf berarah $G = (V, E)$ di mana setiap nilai adalah simpul dan setiap operasi valid adalah busur berbobot 1. Menghitung $f(n)$ berarti mencari jalur terpendek dari simpul n ke simpul 1 dalam graf tersebut.

4.4 Pemodelan DP

Spesifikasi lengkap solusi DP adalah sebagai berikut:

- **State:** $DP[i] = f(i)$ = jumlah langkah minimum dari i ke 1
- **Basis:** $DP[1] = 0, DP[2] = 1$
- **Transisi:** untuk $i \geq 3$:

Relasi Rekurensi MST1

$$DP[i] = \begin{cases} 1 + \min(DP[i/3], DP[i/2], DP[i-1]) & \text{jika } 2 \mid i \text{ dan } 3 \mid i \\ 1 + \min(DP[i/2], DP[i-1]) & \text{jika } 2 \mid i \text{ dan } 3 \nmid i \\ 1 + DP[i/3] & \text{jika } 2 \nmid i \text{ dan } 3 \mid i \\ 1 + DP[i-1] & \text{jika } 2 \nmid i \text{ dan } 3 \nmid i \end{cases}$$

Catatan. Pada kasus $3 \mid i$ dan $2 \nmid i$, implementasi hanya mengambil $1 + DP[i/3]$ tanpa dibandingkan dengan $DP[i-1]$. Hal ini valid karena untuk $i \geq 4$, selalu berlaku $i/3 < i-1$, sehingga membagi tiga selalu mengantarkan ke simpul yang lebih dekat ke 1 dibandingkan mengurangi satu.

4.5 Pendekatan *Hybrid: Bottom-Up + Top-Down*

4.5.1 Motivasi Pendekatan *Hybrid*

Soal ini memiliki batasan $0 < N \leq 2 \times 10^7$. Mengalokasikan array DP berukuran penuh membutuhkan $2 \times 10^7 \times 4 \text{ byte} = 80 \text{ MB}$, yang berpotensi melampaui batas memori. Solusinya adalah membatasi tabel DP hingga $n = 50.000$ dan mengandalkan rekursi untuk nilai yang lebih besar.

4.5.2 Keamanan Rekursi untuk $n > 50.000$

Untuk $n > 50.000$, operasi pembagian ($n/2$ atau $n/3$) langsung membawa n ke rentang yang sudah tersedia dalam tabel DP. Kedalaman rekursi maksimum adalah $O(\log n)$, sehingga tidak ada risiko *stack overflow*. Ini adalah kunci keamanan pendekatan *hybrid*: batas 50.000 dipilih tepat agar selalu ada entri tabel yang dapat diakses secara langsung.

4.5.3 Implementasi C++ yang Disubmit

```

1  #include <cstdio>
2  #include <algorithm>
3  using namespace std;
4
5  int DP[50005];
6
7  int f(int n) {
8      if (n <= 50000) return DP[n];
9      else if (n%2==0 && n%3!=0) return 1 + min(f(n/2), f(n-1));
10     else if (n%3==0 && n%2!=0) return 1 + f(n/3);
11     else if (n%3==0 && n%2==0) return 1 + min(f(n/3), min(f(n/2), f(n-1)));
12     else return 1 + f(n-1);
13 }
14
15 int main() {
16     int T, n, cases = 1;
17     DP[1] = 0; DP[2] = 1;
18     for (int i = 3; i <= 50000; i++) {
19         if (i%2==0 && i%3!=0) DP[i] = 1 + min(DP[i/2], DP[i-1]);
20         else if (i%3==0 && i%2!=0) DP[i] = 1 + DP[i/3];
21         else if (i%2==0 && i%3==0) DP[i] = 1 + min(DP[i/3], min(DP[i/2], DP[i-1]));
22         else DP[i] = 1 + DP[i-1];
23     }
24     scanf("%d", &T);
25     while (T--) {
26         scanf("%d", &n);
27         printf("Case %d: %d\n", cases, f(n));
28         cases++;
29     }

```

```

30     return 0;
31 }

```

Listing 3: Solusi SPOJ MST1 — Pendekatan *Hybrid* (kode submit)

4.6 Analisis Kompleksitas

Tabel 2: Analisis Kompleksitas MST1 — Pendekatan *Hybrid*

Fase	Kompleksitas
<i>Precompute</i> (<i>bottom-up</i>)	$O(B)$ waktu, $O(B)$ memori ($B = 50.000$)
Kueri $n \leq B$	$O(1)$
Kueri $n > B$	$O(\log n)$ waktu, $O(\log n)$ <i>stack</i>
Total (T kueri)	$O(B + T \log N_{\max})$

Kekuatan Pendekatan *Hybrid*

Solusi *hybrid* ini mengeksploitasi dua sifat masalah secara simultan: (1) *overlapping subproblems* — tabel DP untuk nilai kecil menghindari komputasi ulang; dan (2) reduksi eksponensial — untuk nilai besar, operasi $n/2$ dan $n/3$ mereduksi masalah secara eksponensial sehingga rekursi memiliki kedalaman $O(\log n)$.

5 Perbandingan dan Analisis Dua Studi Kasus

5.1 Tabel Komparatif

Tabel 3: Perbandingan Menyeluruh EIDSALAMI dan MST1

Aspek	EIDSALAMI	MST1
Paradigma utama	Matematika / <i>Closed-form</i>	<i>Dynamic Programming</i>
Struktur masalah	Rekurensi kombinatorial	Transisi keadaan
Ruang <i>state</i>	$O(N \times K)$	$O(N)$
Strategi optimasi	Formula eksak atau DP tabulasi	<i>Memoization</i> tabel
Pola solusi	Periodik (trigonometri)	Rekursi + tabel
Risiko utama	<i>Floating-point error</i>	Kedalaman <i>stack</i>
Kompleksitas total	$O(NK)$ precompute, $O(1)$ kueri	$O(B + T \log N)$

5.2 Narasi Perbandingan

Meski sekilas sangat berbeda, EIDSALAMI dan MST1 berbagi satu tema fundamental: keduanya adalah masalah rekurensi yang memerlukan strategi untuk menghindari komputasi redundan. Perbedaannya terletak pada dimensi di mana optimasi dilakukan.

Pada **EIDSALAMI**, optimasi berada pada dimensi matematis. Rekurensi dua-dimensi $P(n, k)$ dapat disederhanakan menjadi formula satu-dimensi dengan mengeksploitasi struktur aljabar persamaan karakteristik. Pendekatan ini memindahkan beban komputasi dari *runtime* ke tahap derivasi matematis *offline*.

Pada **MST1**, optimasi berada pada dimensi komputasi. Tidak ada formula matematis eksak yang dapat ditemukan; alih-alih, solusi bergantung sepenuhnya pada tabel DP dan rekursi terstruktur. Ini adalah rekayasa algoritmik murni yang memanfaatkan struktur transisi masalah.

Tabel 4: Perbandingan Paradigma Pemodelan Matematis vs. Pemrograman Dinamis

Aspek	Pemodelan Matematis	Pemrograman Dinamis
Cocok untuk	Struktur aljabar kaya, rekurensi linear, pola periodik	Substruktur optimal jelas, tidak ada pola aljabar yang mudah ditemukan
Hasil akhir	Formula $O(1)$	Tabel $O(N)$
Risiko utama	Galat numerik (<i>floating</i>)	Penggunaan memori dan kedalaman rekursi
Beban derivasi	Tinggi (analisis manual)	Sistematis (dibantu komputer)

6 Kesimpulan

6.1 Pola Berpikir Sistematis dalam Memecahkan Masalah DP

Pertemuan ketujuh ini memperlihatkan bahwa penyelesaian masalah algoritmik yang baik mengikuti hierarki penalaran yang konsisten. Berikut adalah alur yang terbentuk dari kedua studi kasus:

1. **Identifikasi sifat DP.** Pastikan masalah memiliki *overlapping subproblems* dan *optimal substructure* sebelum menerapkan DP.
2. **Definisi *state* secara presisi.** Tentukan secara eksplisit apa yang direpresentasikan oleh $dp[i]$, termasuk apakah diperlukan lebih dari satu dimensi.
3. **Penurunan rekurensi.** Rumuskan transisi antar *state* berdasarkan seluruh pilihan yang tersedia, bukan hanya yang terlihat paling menguntungkan secara lokal.
4. **Penentuan basis.** Identifikasi nilai-nilai awal dan kondisi tepi yang menjadi fondasi seluruh rekurensi.
5. **Verifikasi manual.** Selalu periksa rekurensi dengan tabulasi kasus kecil sebelum membuat kode. Langkah ini mencegah kesalahan logika yang tidak terdeteksi.
6. **Reduksi *state*.** Jika memungkinkan, sederhanakan DP multi-dimensi ke bentuk yang lebih efisien melalui substitusi aljabar atau eksplorasi *closed-form*.
7. **Penanganan numerik.** Gunakan bilangan bulat eksak bila memungkinkan; untuk *closed-form*, waspadai potensi *floating-point error* pada nilai besar.

Alur Berpikir Umum Soal DP

Pahami Soal → Definisikan *State* → Turunkan Rekurensi → Tentukan Basis →
Verifikasi Manual → Implementasi Efisien

6.2 Ringkasan Konsep Kunci

Berdasarkan analisis kedua studi kasus, terdapat sejumlah konsep kunci yang harus dikuasai:

1. **Rekurensi adalah jembatan universal.** Baik EIDSALAMI maupun MST1 dimulai dari rekurensi. Rekurensi bukan hanya alat untuk implementasi DP, tetapi juga titik awal yang sah untuk derivasi *closed-form*. Kemampuan memformulasikan rekurensi yang benar adalah keterampilan paling fundamental dalam pemrograman kompetitif.
2. ***Closed-form* adalah imbalan atas pemahaman mendalam.** Formula $P(n, 3)$ yang mengandung fungsi kosinus bukan sekadar trik matematika — ia adalah konsekuensi logis dari struktur masalah yang dieksploitasi melalui teori fungsi pembangkit. Pemahaman tentang *mengapa* kosinus muncul memungkinkan generalisasi ke masalah-masalah serupa.
3. **Rekurensi partisi $P(n, k)$.** Formula $P(n, k) = P(n - 1, k - 1) + P(n - k, k)$ diturunkan dari klasifikasi bagian terkecil, bukan dari hafalan. Memahami derivasinya membuat rumus ini mudah direkonstruksi kapan pun dibutuhkan.
4. **Kegagalan *greedy* pada MST1.** Ketiadaan *greedy choice property* menjadi alasan mendasar mengapa strategi serakah tidak bekerja. DP menjamin optimalitas justru karena ia mempertimbangkan seluruh alternatif, bukan hanya yang tampak terbaik secara lokal.
5. **Pendekatan *hybrid* sebagai keterampilan rekayasa.** Solusi MST1 yang menggabungkan *bottom-up* DP dengan rekursi *top-down* menunjukkan bahwa solusi optimal tidak harus murni satu paradigma. Kemampuan mengidentifikasi batas yang tepat dan mengombinasikan pendekatan adalah keterampilan tingkat lanjut.
6. ***Trade-off* bilangan bulat vs. *floating-point*.** Pilihan antara *closed-form* (lebih cepat, rentan galat) dan DP (lebih lambat, akurat eksak) harus mempertimbangkan batasan soal dan karakteristik nilai keluarannya.
7. **Pemodelan masalah sebagai graf.** Memodelkan MST1 sebagai masalah *shortest-path* membuka perspektif baru dan memungkinkan penerapan algoritma BFS atau Dijkstra sebagai alternatif. Kebiasaan memodelkan masalah secara abstrak adalah investasi jangka panjang dalam kemampuan algoritmik.

Pesan Utama Pertemuan 7

DP bukan tentang menghafal rumus. Pertemuan ini menunjukkan bahwa **pe-milihan paradigma yang tepat** — apakah matematis-kombinatorial atau optimasi berbasis *state* — menentukan kemudahan derivasi, efisiensi memori, dan keamanan numerik. Selalu verifikasi rekurensi dengan tabulasi manual sebelum membuat kode.

7 Referensi

Pustaka

- [1] Rosen, K. H. (2012). *Discrete Mathematics and Its Applications* (7th ed.). McGraw-Hill. Bab 8: Linear Recurrence Relations.
- [2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [3] SPOJ. (n.d.). *EIDSALAMI – Eid Salami*. <https://www.spoj.com/problems/EIDSALAMI/>
- [4] SPOJ. (n.d.). *MST1 – Minimum Step To One*. <https://www.spoj.com/problems/MST1/>
- [5] Sulaiman, R. (2026). *Materi Kuliah PAA Pertemuan Ke-7*. Institut Teknologi Sepuluh Nopember.

8 Bukti Submission

GilbranNRP134: submissions
Eid Salami

ID	DATE	PROBLEM	RESULT	TIME	MEM	LANG
35715999	2026-04-28 18:51:58	Eid Salami	accepted edit ideone it	0.01	5.2M	CPP14

Gambar 1: Accepted submission — SPOJ EIDSALAMI (Eid Salami)

GilbranNRP134: submissions
Minimum Step To One

ID	DATE	PROBLEM	RESULT	TIME	MEM	LANG
35716009	2026-04-28 16:59:09	Minimum Step To One	accepted edit ideone it	0.01	5.2M	CPP14

Gambar 2: Accepted submission — SPOJ MST1 (Minimum Step To One)